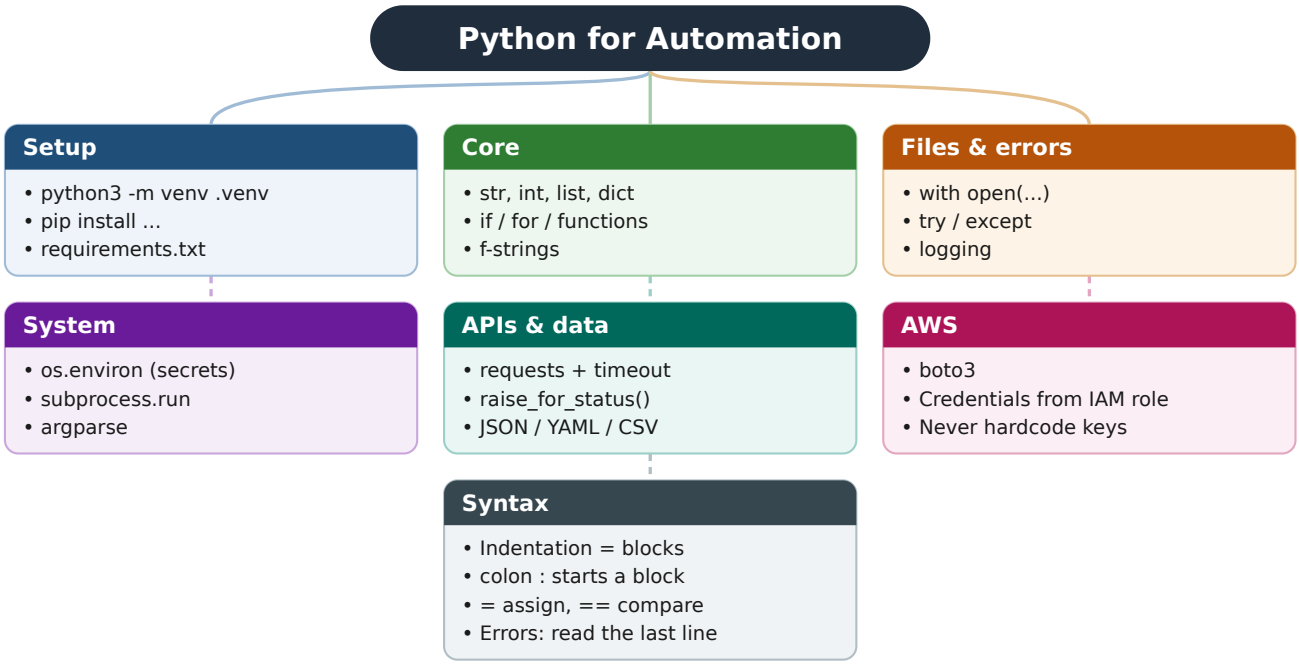
6. Python for DevOps Automation

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 6 of 18 · Thirupathi Keloth · v2.0

Contents

6.0 Why Python for DevOps
6.1 Setup and environment
6.2 Syntax basics
6.3 Data types
6.4 Collections in depth
6.5 Control flow
6.6 Functions
6.7 Modules, packages and the standard library
6.8 Files, paths and data formats
6.9 Errors and exceptions
6.10 Logging
6.11 Running system commands: subprocess
6.12 HTTP and REST APIs with requests
6.13 AWS automation with boto3
6.14 Other libraries DevOps engineers use
6.15 Classes and dataclasses (enough to read and write tools)
6.16 Concurrency: checking many things fast
6.17 Testing with pytest
6.18 Code quality and packaging
6.19 Project: service health checker CLI
6.20 Common errors and what they mean
6.21 Interview questions and answers
6.22 Self-check: I can...



6.0 Why Python for DevOps

Python is the most used language for automation: AWS (boto3), Kubernetes and Docker SDKs, Ansible (written in Python), API integrations, data processing, CLIs, Lambda functions, monitoring checks. It's readable, has a huge standard library and packages for everything. Use **Bash** for short command glue; use **Python** when you need logic, data structures, error handling, APIs, tests or cross-platform tools.

6.1 Setup and environment

Task	Command
Check version	`python3 --version` (use 3.10+)
Interactive shell (REPL)	`python3` (exit with `exit()` or Ctrl+D)
Run a script	`python3 script.py`
Create a virtual environment	`python3 -m venv .venv`
Activate it	`source .venv/bin/activate` (Windows: `.venv\Scripts\activate`)
Install packages	`pip install requests boto3 pyyaml`
Freeze exact versions	`pip freeze > requirements.txt`
Reproduce	`pip install -r requirements.txt`
Install CLI tools globally, isolated	`pipx install <tool>`
Deactivate	`deactivate`

Why virtual environments: each project gets its own packages and versions; no conflicts with system Python (which the OS itself uses: never `sudo pip install` into it). Add `.venv/` to `.gitignore`. Modern projects describe dependencies in `pyproject.toml`; tools like **uv**, Poetry or pip-tools manage lock files.

6.2 Syntax basics

```
# Comments start with #
name = "web01"            # variables: no type declaration, snake_case names
MAX_RETRIES = 3           # constants: UPPER_CASE by convention

if name.startswith("web"):
    print("web server")   # indentation (4 spaces) defines blocks; a colon starts a block
else:
    print("other")

print(f"Server {name} allows {MAX_RETRIES} retries")   # f-string formatting
```

- **Indentation is syntax**: mixing tabs/spaces or wrong indentation → `IndentationError`.
- Everything is an object; variables are names pointing to objects.
- Style guide: **PEP 8** (4-space indents, snake_case functions/variables, CapWords classes, max ~88–100 chars per line with tools like black/ruff).

6.3 Data types

Type	Example	Mutable?	Notes
`int`	`42`, `1_000_000`	No	Unlimited size
`float`	`3.14`, `1e-3`	No	Binary floating point (`0.1 + 0.2 != 0.3` exactly)
`bool`	`True`, `False`	No	Subclass of int
`str`	`"text"`, `'text'`, `"""multi-line"""`	No	Unicode text

Type	Example	Mutable?	Notes
`bytes`	`b"raw"`	No	Binary data (`.encode()` / `.decode()`)
`None`	`None`	—	"No value"
`list`	`[1, 2, 3]`	**Yes**	Ordered
`tuple`	`(1, 2)`	No	Ordered, fixed
`dict`	`{"env": "dev"}`	**Yes**	Key → value, insertion-ordered
`set`	`{"a", "b"}`	**Yes**	Unique items, fast membership

Truthiness: `False` , `None` , `0` , `""` , `[]` , `{}` , `set()` are false in conditions; everything else is true. `if not items:` = "list is empty".

Conversions: `int("80")` , `float("1.5")` , `str(80)` , `bool("")` , `list("abc")` , `tuple(lst)` , `set(lst)` , `dict(pairs)` . Check type: `type(x)` , `isinstance(x, str)` .

Strings

```
s = "  Web-Server-01  "
s.strip()                       # 'Web-Server-01'
s.strip().lower()               # 'web-server-01'
"a,b,c".split(",")              # ['a', 'b', 'c']
"-".join(["2026", "10", "17"]) # '2026-10-17'
"error" in "fatal error"        # True
"app.log".endswith(".log")      # True
"web01".replace("01", "02")     # 'web02'
"web01"[0:3], "web01"[-2:]      # ('web', '01')  slicing [start:stop]
len("web01")                    # 5
"Port: {}".format(80)           # older formatting
f"{3.14159:.2f} | {42:>6} | {'ok':<4}|"   # '3.14 |     42 | ok  |'
r"C:\new\folder"                # raw string: backslashes not escapes (regex, Windows paths)
```

6.4 Collections in depth

Lists

```
servers = ["web1", "web2"]
servers.append("web3")              # add one
servers.extend(["db1", "db2"])      # add many
servers.insert(0, "lb1")            # insert at position
servers.remove("db2")               # remove by value (ValueError if missing)
last = servers.pop()                # remove and return last
servers.sort()                      # sort in place
sorted(servers, reverse=True)       # new sorted list
servers.index("web2")               # position
servers.count("web1")               # occurrences
servers[0], servers[-1], servers[1:3]   # first, last, slice
```

Dictionaries

```
server = {"name": "web1", "port": 80, "tags": ["prod"]}
server["port"]                        # 80 (KeyError if missing)
server.get("env", "dev")              # 'dev' (safe default)
server["env"] = "prod"                # add/update
server.update({"port": 8080, "az": "a"})
del server["az"]
"name" in server                      # key exists?
for key, value in server.items():     # iterate pairs
    print(key, value)
list(server.keys()), list(server.values())
config = {**server, "port": 443}      # merge (new dict)
```

Sets and tuples

```
allowed = {"dev", "staging", "prod"}
"qa" in allowed                     # fast membership → False
{"a", "b"} | {"b", "c"}             # union {'a','b','c'}
{"a", "b"} & {"b", "c"}             # intersection {'b'}
{"a", "b"} - {"b"}                  # difference {'a'}
host, port = ("db.internal", 5432) # tuple unpacking
```

Comprehensions and useful built-ins

```
ports = [80, 443, 8080, 22]
web_ports = [p for p in ports if p != 22]                # list comprehension
squares = {p: p * p for p in range(3)}                   # dict comprehension
unique_envs = {s.split("-")[0] for s in ["prod-a", "prod-b", "dev-a"]}   # set → {'prod', 'dev'}

for i, name in enumerate(["web1", "web2"], start=1):      # index + value
    print(i, name)
for name, ip in zip(["web1", "web2"], ["10.0.1.5", "10.0.1.6"]):   # pair two lists
    print(name, ip)

instances = [{"id": "i-1", "cpu": 70}, {"id": "i-2", "cpu": 95}]
busiest = max(instances, key=lambda i: i["cpu"])           # {'id': 'i-2', 'cpu': 95}
by_cpu = sorted(instances, key=lambda i: i["cpu"], reverse=True)
any(i["cpu"] > 90 for i in instances), all(i["cpu"] > 50 for i in instances)   # (True, True)
sum(i["cpu"] for i in instances) / len(instances)          # 82.5
```

6.5 Control flow

```
cpu = 85
if cpu >= 90:
    level = "critical"
elif cpu >= 70:
    level = "warning"
else:
    level = "ok"

status = "up" if cpu < 100 else "down"     # conditional expression

for attempt in range(1, 4):                # 1, 2, 3
    if attempt == 2:
        continue                           # skip to next iteration
    print("attempt", attempt)
else:
    print("loop finished without break")   # for-else: runs if no break

retries = 0
while retries < 3:
    retries += 1

match level:                               # structural pattern matching (Python 3.10+)
    case "critical":
        print("page on-call")
    case "warning":
        print("create ticket")
    case _:
        print("nothing to do")
```

6.6 Functions

```
def check_port(host: str, port: int = 22, *, timeout: float = 3.0) -> bool:
    """Return True if a TCP connection to host:port succeeds.

    timeout is keyword-only (after the *), e.g. check_port("db", 5432, timeout=1).
    """
    import socket
    try:
        with socket.create_connection((host, port), timeout=timeout):
            return True
    except OSError:
        return False


def summarize(*hosts: str, **labels: str) -> dict:
    """*args collects positional args into a tuple; **kwargs collects keyword args into a dict."""
    return {"count": len(hosts), "hosts": hosts, "labels": labels}


print(summarize("web1", "web2", env="prod", team="shop"))
# {'count': 2, 'hosts': ('web1', 'web2'), 'labels': {'env': 'prod', 'team': 'shop'}}


def min_max(values: list[int]) -> tuple[int, int]:
    return min(values), max(values)        # returns a tuple


low, high = min_max([3, 9, 1])             # 1, 9
```

Concept	Detail
Default values	`port: int = 22`. Never use a mutable default (`def f(items=[])`): use `None` and create inside
Type hints	Documentation + checking with mypy; not enforced at runtime
Docstring	First string in the function: shown by `help()`
Scope	Variables assigned inside a function are local; avoid `global`
Lambda	Small anonymous function: `key=lambda x: x["cpu"]`
Return	No `return` → returns `None`

6.7 Modules, packages and the standard library

```
import os                                   # whole module
import json as js                           # alias
from pathlib import Path                    # one name
from datetime import datetime, timedelta, timezone
```

A **module** is a `.py` file; a **package** is a folder of modules (with `__init__.py`). Your own `utils.py` can be imported with `import utils` from the same folder. **Don't name your files after standard modules** (`json.py`, `logging.py`): they shadow the real ones.

Standard library module	Use in DevOps
`os`, `sys`	Environment variables, exit codes, arguments, platform
`pathlib`, `shutil`, `glob`, `tempfile`	Paths, copying/moving, patterns, temp files
`subprocess`	Run commands
`json`, `csv`, `configparser`, `tomllib`	Data formats
`re`	Regular expressions
`datetime`, `time`	Timestamps, durations, sleeping
`logging`	Proper logs
`argparse`	Command-line interfaces
`socket`, `ssl`, `urllib`	Networking basics
`hashlib`, `secrets`, `base64`	Checksums, secure random tokens, encoding
`collections` (`Counter`, `defaultdict`, `deque`)	Counting, grouping
`concurrent.futures`	Parallel work
`platform`, `getpass`, `shlex`	System info, hidden input, safe shell splitting

```
from collections import Counter, defaultdict

codes = ["200", "500", "200", "404", "200"]
Counter(codes).most_common(2)                  # [('200', 3), ('500', 1)]

by_team = defaultdict(list)
for svc, team in [("api", "shop"), ("web", "shop"), ("auth", "core")]:
    by_team[team].append(svc)                  # {'shop': ['api', 'web'], 'core': ['auth']}

from datetime import datetime, timedelta, timezone
now = datetime.now(timezone.utc)
cutoff = now - timedelta(days=7)
print(now.isoformat(), now.strftime("%Y-%m-%d %H:%M"))
datetime.fromisoformat("2026-10-17T09:30:00+00:00") < now     # True
```

6.8 Files, paths and data formats

Files and paths

```
from pathlib import Path

base = Path("/tmp/demo")
base.mkdir(parents=True, exist_ok=True)          # like mkdir -p
report = base / "report.txt"                     # join paths with /
report.write_text("line 1\nline 2\n")
print(report.read_text())
print(report.exists(), report.is_file(), report.suffix, report.stem, report.parent)

with open(report, "a", encoding="utf-8") as f:   # 'with' closes the file automatically
    f.write("line 3\n")                          # modes: r, w (truncate), a (append), b (binary)

with open(report, encoding="utf-8") as f:        # stream large files line by line (low memory)
    for line in f:
        print(line.rstrip())

for log in base.glob("*.txt"):                   # rglob("*.log") searches recursively
    print(log.name, log.stat().st_size)
```

`shutil.copy2(src, dst)`, `shutil.move`, `shutil.rmtree(dir)`, `shutil.disk_usage("/")`, `tempfile.TemporaryDirectory()`.

JSON

```
import json

data = {"service": "api", "replicas": 3, "ports": [80, 443]}
text = json.dumps(data, indent=2)        # dict → JSON string
again = json.loads(text)                 # JSON string → dict

with open("/tmp/demo/config.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)         # write to file
with open("/tmp/demo/config.json", encoding="utf-8") as f:
    cfg = json.load(f)                   # read from file
print(cfg["ports"][1])                   # 443
```

YAML, CSV, INI/TOML

```
import csv
import yaml                              # pip install pyyaml

doc = yaml.safe_load("""
service: api
replicas: 3
env:
  - name: LOG_LEVEL
    value: info
""")                                     # ALWAYS safe_load (yaml.load can execute code)
print(doc["env"][0]["value"])            # info
print(yaml.safe_dump(doc, sort_keys=False))

with open("/tmp/demo/users.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["user", "team"])
    w.writeheader()
    w.writerows([{"user": "alice", "team": "ops"}, {"user": "bob", "team": "dev"}])

with open("/tmp/demo/users.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["user"], row["team"])
```

`configparser` reads INI files; `tomllib` (3.11+) reads TOML (`pyproject.toml`).

Environment variables and configuration

```
import os

region = os.environ.get("AWS_REGION", "ap-south-1")    # optional with default
token = os.environ["API_TOKEN"]                        # required: KeyError if missing
debug = os.getenv("DEBUG", "false").lower() == "true"  # env vars are always strings
```

Configuration order of precedence (common pattern): defaults in code → config file (YAML) → environment variables → command-line arguments. **Secrets** come from environment variables injected by the platform (CI secrets, Kubernetes Secrets, ECS task definitions) or directly from a secrets manager (AWS Secrets Manager, Vault). Never commit them; `python-dotenv` can load a local, git-ignored `.env` file for development.

6.9 Errors and exceptions

```
import sys


class ConfigError(Exception):
    """Raised when configuration is invalid."""


def load_port(value: str) -> int:
    try:
        port = int(value)
    except ValueError as e:
        raise ConfigError(f"port must be a number, got {value!r}") from e
    if not 1 <= port <= 65535:
        raise ConfigError(f"port out of range: {port}")
    return port


try:
    port = load_port("80a")
except ConfigError as e:
    print(f"configuration error: {e}", file=sys.stderr)
    port = 8080
else:
    print("valid port")                    # runs only if no exception
finally:
    print("always runs (cleanup)")
```

Practice	Why
Catch **specific** exceptions (`FileNotFoundError` , `KeyError` , `requests.Timeout`)	A bare `except:` hides bugs and even Ctrl+C
Add context, re-raise with `from e`	Keeps the original traceback
Fail fast on bad config	Clear error at startup instead of strange behaviour later
Exit with codes: `sys.exit(0)` success, `sys.exit(1)` failure	CI, cron and systemd rely on them
Read tracebacks **bottom-up**	The last line is the error; the lines above show where it happened

6.10 Logging

```
import logging

logging.basicConfig(
    level=logging.INFO,                    # DEBUG < INFO < WARNING < ERROR < CRITICAL
    format="%(asctime)s %(levelname)s %(name)s: %(message)s",
)
log = logging.getLogger("healthcheck")

log.debug("not shown at INFO level")
log.info("checking %s", "web1")                          # lazy formatting
log.warning("slow response: %.2fs", 2.5)
try:
    1 / 0
except ZeroDivisionError:
    log.exception("calculation failed")                  # logs the traceback
```

- Use `logging`, not `print`, in tools that run unattended.
- Log to **stdout/stderr** in containers (the platform collects it); to files with `RotatingFileHandler` on servers.
- **Structured (JSON) logs** are easier to search in CloudWatch/Loki/Elastic (libraries: `python-json-logger`, `structlog`).
- Never log secrets or full tokens.

6.11 Running system commands: subprocess

```
import shlex
import subprocess

result = subprocess.run(
    ["df", "-h", "/"],          # list of arguments: no shell, no injection risk
    capture_output=True,        # capture stdout/stderr
    text=True,                  # str instead of bytes
    check=True,                 # raise CalledProcessError on non-zero exit
    timeout=30,                 # raise TimeoutExpired if it hangs
)
print(result.stdout)

r = subprocess.run(["systemctl", "is-active", "nginx"], capture_output=True, text=True)
print("nginx:", r.stdout.strip(), "exit", r.returncode)   # no check=True → inspect yourself

cmd = shlex.split("git log --oneline -n 3")               # split a string safely
print(subprocess.run(cmd, capture_output=True, text=True).stdout)
```

Avoid `shell=True` with any user input (command injection). If you need shell features, build the pipeline in Python or pass a fixed string you fully control.

6.12 HTTP and REST APIs with requests

```
import os

import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()                                    # reuses connections
retries = Retry(total=3, backoff_factor=1, status_forcelist=[429, 500, 502, 503, 504])
session.mount("https://", HTTPAdapter(max_retries=retries))
session.headers.update({"Accept": "application/vnd.github+json"})
token = os.getenv("GITHUB_TOKEN")                               # optional; raises rate limits
if token:
    session.headers["Authorization"] = f"Bearer {token}"


def list_repos(org: str) -> list[dict]:
    """Return all repositories of an organization, following pagination."""
    repos, page = [], 1
    while True:
        resp = session.get(
            f"https://api.github.com/orgs/{org}/repos",
            params={"per_page": 100, "page": page},
            timeout=10,                                         # ALWAYS set a timeout
        )
        resp.raise_for_status()                                 # error on 4xx/5xx
        batch = resp.json()
        if not batch:
            return repos
        repos.extend(batch)
        page += 1


if __name__ == "__main__":
    repos = sorted(list_repos("kubernetes"), key=lambda r: r["stargazers_count"], reverse=True)
    for repo in repos[:5]:
        print(f'{repo["name"]:<30} {repo["stargazers_count"]:>7}')
```

Call	Use
`session.get(url, params=..., timeout=10)`	Read
`session.post(url, json={...})`	Create (sends JSON body + header)
`session.put / patch / delete`	Update / delete
`resp.status_code`, `resp.json()`, `resp.text`, `resp.headers`	Read the response
`resp.raise_for_status()`	Turn HTTP errors into exceptions
Auth	`headers={"Authorization": f"Bearer {token}"}`, `auth=(user, password)`

API etiquette: handle **pagination**, respect **rate limits** (`429`, `Retry-After`/rate-limit headers), retry only idempotent calls or use idempotency keys, set timeouts, keep tokens in env/secret stores.

6.13 AWS automation with boto3

```
import boto3
from botocore.exceptions import ClientError

# credentials come from env vars, a profile/SSO, or an IAM role - never from code
session = boto3.Session(region_name="ap-south-1")
ec2 = session.client("ec2")

# Paginators handle pages for you
paginator = ec2.get_paginator("describe_instances")
pages = paginator.paginate(Filters=[{"Name": "instance-state-name", "Values": ["running"]}])
for page in pages:
    for reservation in page["Reservations"]:
        for inst in reservation["Instances"]:
            tags = {t["Key"]: t["Value"] for t in inst.get("Tags", [])}
            print(inst["InstanceId"], inst["InstanceType"],
                  tags.get("Name", "-"), tags.get("Owner", "MISSING"))

# Error handling
s3 = session.client("s3")
try:
    s3.head_bucket(Bucket="my-bucket-that-may-not-exist")
except ClientError as e:
    code = e.response["Error"]["Code"]                  # e.g. '404', '403', 'AccessDenied'
    print(f"bucket check failed: {code}")

# Upload / download
s3.upload_file("/tmp/demo/report.txt", "my-reports-bucket", "reports/report.txt")

# Read a secret parameter (decrypted)
ssm = session.client("ssm")
param = ssm.get_parameter(Name="/shop/prod/db/password", WithDecryption=True)
db_password = param["Parameter"]["Value"]

# Waiters: block until a state is reached
ec2.get_waiter("instance_stopped").wait(InstanceIds=["i-0123456789abcdef0"])
```

Concept	Detail
Client vs resource	`client` = low-level, maps 1:1 to API calls (recommended); `resource` = older object-oriented API
Credentials chain	Same as the AWS CLI: env vars → shared config/SSO profile → container/instance role. **Never hard-code keys**
Paginators	Many list/describe calls return pages; use `get_paginator`
Waiters	Wait for states (instance running, stack complete)
Errors	`botocore.exceptions.ClientError`; check `e.response["Error"]["Code"]`; throttling → retries (configurable via `botocore.config.Config(retries=...)`)
Dry run	EC2 calls accept `DryRun=True` to test permissions
Testing	`moto` library mocks AWS for unit tests

6.14 Other libraries DevOps engineers use

Library	Use
`kubernetes`	Kubernetes API client (list pods, patch deployments)

Library	Use
`docker`	Docker Engine API
`paramiko`, `fabric`	SSH automation
`jinja2`	Templates (config files; Ansible uses it)
`click` / `typer`	Friendly CLIs with subcommands
`rich`	Pretty terminal tables/progress
`pydantic`	Validate config/data with type hints
`prometheus_client`	Expose custom metrics
`pytest`, `moto`, `responses`	Testing, AWS mocks, HTTP mocks
`black`, `ruff`, `mypy`	Formatting, linting, type checking

6.15 Classes and dataclasses (enough to read and write tools)

```
from dataclasses import dataclass, field


@dataclass
class Service:
    name: str
    url: str
    timeout: float = 5.0
    tags: list[str] = field(default_factory=list)   # mutable default done right

    def label(self) -> str:
        return f"{self.name} ({self.url})"


svc = Service("api", "https://api.example.com/health", tags=["prod"])
print(svc.label(), svc.timeout)                      # api (https://api.example.com/health) 5.0
```

A **class** bundles data (attributes) and behaviour (methods); `self` is the instance. `@dataclass` generates `__init__`, `__repr__` and comparisons automatically: ideal for config and result objects.

6.16 Concurrency: checking many things fast

Network checks spend most of their time waiting, so threads speed them up a lot.

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import socket


def port_open(host: str, port: int) -> tuple[str, int, bool]:
    try:
        with socket.create_connection((host, port), timeout=2):
            return host, port, True
    except OSError:
        return host, port, False


targets = [("github.com", 443), ("example.com", 80), ("10.255.255.1", 22)]
with ThreadPoolExecutor(max_workers=10) as pool:
    futures = [pool.submit(port_open, h, p) for h, p in targets]
    for fut in as_completed(futures):
        host, port, ok = fut.result()
        print(f"{host}:{port} {'open' if ok else 'closed/unreachable'}")
```

Use `ProcessPoolExecutor` for CPU-heavy work; `asyncio` (+ `httpx` / `aiohttp`) for thousands of concurrent I/O tasks.

6.17 Testing with pytest

```
# file: checks.py
def classify_cpu(cpu: float) -> str:
    if cpu >= 90:
        return "critical"
    if cpu >= 70:
        return "warning"
    return "ok"
```

```
# file: test_checks.py   (run: pytest -q)
import pytest
from checks import classify_cpu


@pytest.mark.parametrize("cpu, expected", [(10, "ok"), (70, "warning"), (95, "critical")])
def test_classify_cpu(cpu, expected):
    assert classify_cpu(cpu) == expected
```

Mocking external calls so tests don't hit real APIs:

```
from unittest.mock import patch, MagicMock
import requests


def get_status(url: str) -> int:
    return requests.get(url, timeout=5).status_code


def test_get_status():
    fake = MagicMock(status_code=503)
    with patch("requests.get", return_value=fake) as mocked:
        assert get_status("https://example.com") == 503
        mocked.assert_called_once()
```

Run tests in CI on every pull request (`pytest -q --maxfail=1`), together with `ruff check .` and `black --check .`.

6.18 Code quality and packaging

Practice	Tool / how
Formatting	`black .` (or `ruff format`)
Linting	`ruff check .` (fast, replaces flake8/isort/many plugins)
Type checking	`mypy .` with type hints
Dependencies pinned	`requirements.txt` with versions or a lock file (uv/poetry/pip-tools)
Security scanning	`pip-audit`, Dependabot
Entry point	`if __name__ == "__main__": main()` and `sys.exit(main())`
CLI	`argparse` (stdlib) or `click` / `typer`
Distribution	Container image (`python:3.12-slim`) or a package with a console script in `pyproject.toml`
Project layout	`src/<pkg>/`, `tests/`, `pyproject.toml`, `README.md`, `.gitignore`

6.19 Project: service health checker CLI

A complete, realistic tool combining everything: YAML config, environment overrides, parallel HTTP checks, logging, JSON report and exit codes for CI/cron.

```
# services.yaml
timeout: 5
services:
  - name: github
    url: https://api.github.com
  - name: example
    url: https://example.com
    expect_status: 200
```

```
#!/usr/bin/env python3
"""healthcheck.py - check HTTP services listed in a YAML file.

Usage: python3 healthcheck.py services.yaml [--report out.json] [--workers 8]
Exit code: 0 all healthy, 1 at least one unhealthy, 2 configuration error.
"""
import argparse
import json
import logging
import os
import sys
import time
from concurrent.futures import ThreadPoolExecutor
from dataclasses import asdict, dataclass

import requests
import yaml

log = logging.getLogger("healthcheck")


@dataclass
class Result:
    name: str
    url: str
    ok: bool
    status: int | None
    seconds: float
    error: str = ""


def check(name: str, url: str, expect: int, timeout: float) -> Result:
    start = time.monotonic()
    try:
        resp = requests.get(url, timeout=timeout)
        ok = resp.status_code == expect
        return Result(name, url, ok, resp.status_code, round(time.monotonic() - start, 3))
    except requests.RequestException as e:
        return Result(name, url, False, None, round(time.monotonic() - start, 3), str(e))


def load_config(path: str) -> dict:
    try:
        with open(path, encoding="utf-8") as f:
            cfg = yaml.safe_load(f) or {}
    except (OSError, yaml.YAMLError) as e:
        raise SystemExit(f"config error: {e}") from e
    if not cfg.get("services"):
        raise SystemExit("config error: no services defined")
    cfg["timeout"] = float(os.getenv("HC_TIMEOUT", cfg.get("timeout", 5)))
    return cfg


def main() -> int:
    parser = argparse.ArgumentParser(description="HTTP health checker")
    parser.add_argument("config")
    parser.add_argument("--report", help="write a JSON report to this file")
    parser.add_argument("--workers", type=int, default=8)
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                        format="%(asctime)s %(levelname)s %(message)s")

    try:
        cfg = load_config(args.config)
    except SystemExit as e:
```

```
        log.error("%s", e)
        return 2

    with ThreadPoolExecutor(max_workers=args.workers) as pool:
        results = list(pool.map(
            lambda s: check(s["name"], s["url"], s.get("expect_status", 200), cfg["timeout"]),
            cfg["services"]))

    for r in results:
        level = logging.INFO if r.ok else logging.ERROR
        log.log(level, "%-10s %-5s status=%s time=%.3fs %s",
                r.name, "UP" if r.ok else "DOWN", r.status, r.seconds, r.error)

    if args.report:
        with open(args.report, "w", encoding="utf-8") as f:
            json.dump([asdict(r) for r in results], f, indent=2)

    return 0 if all(r.ok for r in results) else 1


if __name__ == "__main__":
    sys.exit(main())
```

Extend it: send a Slack/Teams message on failure (webhook URL from an env var), expose results as Prometheus metrics, run it from cron/a systemd timer/a Kubernetes CronJob, package it in a Docker image, add tests with `responses` to mock HTTP.

6.20 Common errors and what they mean

Error	Meaning / fix
`IndentationError` / `TabError`	Inconsistent indentation: use 4 spaces
`SyntaxError`	Missing colon, bracket or quote; check the line shown (or the line before)
`NameError: name 'x' is not defined`	Typo, or used before assignment/import
`TypeError: can only concatenate str (not "int") to str`	Mixing types: use f-strings or `str()`
`KeyError: 'env'`	Missing dict key: use `.get()` or check `in`
`IndexError: list index out of range`	Accessing past the end of a list
`AttributeError: 'NoneType' object has no attribute ...`	A function returned `None` (often a failed lookup)
`ModuleNotFoundError`	Package not installed in **this** venv (activate it, `pip install`), or file name shadows a module
`FileNotFoundError` / `PermissionError`	Wrong path (relative to the current directory!) / permissions
`json.decoder.JSONDecodeError`	Response wasn't JSON (check status code / HTML error page)
`requests.exceptions.ConnectionError` / `Timeout`	DNS/network/firewall problem / server too slow
`botocore.exceptions.NoCredentialsError`	No AWS credentials found (profile, env, role)
`ClientError: AccessDenied`	IAM permissions (see AWS notes)

Error	Meaning / fix
`UnicodeDecodeError`	Wrong encoding: open files with `encoding="utf-8"` (or `errors="replace"`)

6.21 Interview questions and answers

#	Question	Answer
1	Why use Python instead of Bash?	Better for complex logic, data structures, APIs/JSON, error handling, tests and portability
2	What is a virtual environment?	An isolated Python environment per project with its own packages
3	List vs tuple vs set vs dict?	Ordered mutable / ordered immutable / unique unordered / key-value mapping
4	Mutable vs immutable?	Mutable objects (list, dict, set) can change in place; immutable (str, int, tuple) can't
5	What are `*args` and `**kwargs`?	Collect extra positional arguments (tuple) and keyword arguments (dict)
6	Why avoid mutable default arguments?	The default is created once and shared between calls
7	What does `if __name__ == "__main__":` do?	Runs code only when the file is executed directly, not when imported
8	How do you handle exceptions?	try/except with specific exceptions, else/finally, raise with context, proper exit codes
9	How do you run shell commands safely?	`subprocess.run([...], check=True, capture_output=True, text=True, timeout=...)` without `shell=True`
10	How do you call a REST API reliably?	requests with timeouts, raise_for_status, retries with backoff, pagination, auth from env
11	How does boto3 get credentials?	Same chain as the CLI: env → profile/SSO → container/instance role; never hard-coded
12	What are paginators and waiters in boto3?	Helpers that iterate all pages of results / wait until a resource reaches a state
13	`json.load` vs `json.loads`?	From a file object vs from a string
14	Why `yaml.safe_load`?	`yaml.load` can construct arbitrary objects (security risk)
15	How do you speed up many network checks?	Threads (`ThreadPoolExecutor`) or asyncio
16	How do you test code that calls external APIs?	Mock them (`unittest.mock`, `responses`, `moto` for AWS)
17	What is a list comprehension?	Compact syntax to build a list from an iterable with an optional filter
18	How do you manage dependencies reproducibly?	Pinned requirements/lock file in a venv or container image
19	How should a script report failure to CI/cron?	Non-zero exit code via `sys.exit(1)` plus clear logs
20	What is a dataclass?	A class decorator that generates init/repr/eq for data containers

6.22 Self-check: I can…

- [ ] Set up a project with a virtual environment and pinned requirements.
- [ ] Use all core data types, string methods, slicing and f-strings.
- [ ] Use lists, dicts, sets, tuples, comprehensions, `sorted` / `max` with `key`, `enumerate`, `zip`.
- [ ] Write control flow including `match`, and functions with defaults, keyword-only args, `*args/**kwargs`, type hints.
- [ ] Use the standard library: pathlib, json, csv, datetime, collections, subprocess, logging, argparse.
- [ ] Read/write files and JSON/YAML/CSV safely; read configuration and secrets from the environment.
- [ ] Handle exceptions properly and return meaningful exit codes.
- [ ] Call REST APIs with timeouts, retries, pagination and token auth.
- [ ] Automate AWS with boto3 (paginators, waiters, error codes, no hard-coded keys).
- [ ] Write dataclasses and read simple classes.
- [ ] Parallelize I/O-bound checks with threads.
- [ ] Write pytest tests, including mocks for external services.
- [ ] Keep code formatted, linted, typed and packaged as a CLI or container.
- [ ] Build and extend the health checker project.

> **REMEMBER**
>
> - Three habits: **virtual environment**, **timeout on every network call**, **secrets from the environment**.
> - Return a non-zero exit code on failure so cron/CI notices.